

コマ 15: 複数ファイル・前処理の概念

今日のゴール

`#include "file.h"` や `#define NAME value` を含む入力を扱い、複数 `.c` ファイルをまとめてコンパイルできるようにする。

第 14 回までで、グローバル変数とローカル変数を使った標準的なプログラムはコンパイルできるようになった。この回では、プログラムを複数のファイルに分割できるようにする。

```
/* math_util.h */
int my_abs(int x);
int my_pow(int base, int exp);

/* math_util.c */
int my_abs(int x) {
    if (x < 0) return -x;
    return x;
}

int my_pow(int base, int exp) {
    int result;
    int i;
    result = 1;
    for (i = 0; i < exp; i = i + 1) {
        result = result * base;
    }
    return result;
}

/* main.c */
#include "math_util.h"
#define BASE 2

int main() {
    return my_pow(BASE, 3);
}
```

1 この回で扱う範囲

対象にする機能は次の通り。

種類	例
<code>#include</code>	<code>#include "math_util.h"</code>
<code>#define</code>	<code>#define BASE 2</code>
複数ファイル	<code>python3 mycc.py main.c math_util.c</code>
関数プロトタイプ	<code>int f(int x);</code>

この回では、`#define` は単純な定数置換だけを扱う。関数形式マクロや条件コンパイルは扱わない。

2 前処理とは何か

C コンパイラは、ソースコードの字句解析に入る前に前処理を行う。講義のスキャフォールドに含まれている `preprocess()` 関数がこれにあたる。

前処理が行うのは、主に次の 2 つである。

2.1 `#include` はファイル内容の展開

```
#include "math_util.h"
```

この行は、`math_util.h` の内容で置き換えられる。たとえば `math_util.h` に `int my_abs(int x);` と書いてあれば、前処理後のソースにはその内容が埋め込まれる。

2.2 `#define` はトークン列に入る前の置換

```
#define BASE 2

int main() {
    return BASE + 1;
}
```

前処理後は、`BASE` が `2` に置き換えられる。したがって、コンパイラが受け取るのは `return 2 + 1;` である。

3 ASTを確認する: `#define`

`#define` の置換結果がASTでどう見えるか確認する。

```
python3 scaffold/parse_viewer.py sessions/15_preprocess_multifile/tests/define_min
```

このプログラムの内容は次の通り。

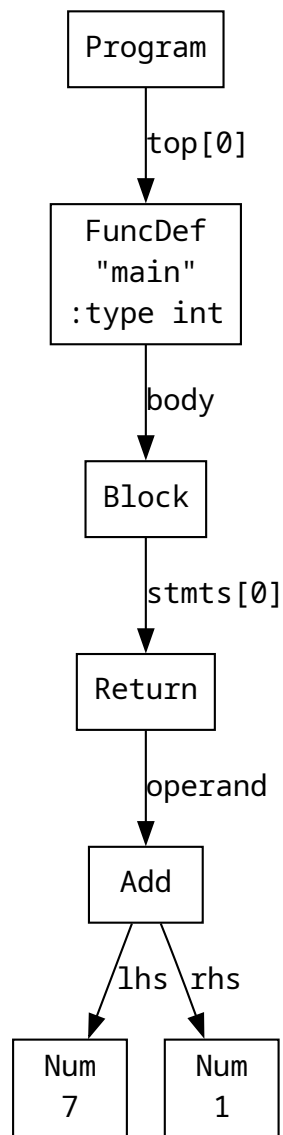
```
#define N 7

int main() {
    return N + 1;
}
```

実行すると、次のようなS式が表示される。

```
(program
  (funcdef "main" :type int (params)
    (block
      (return
        (add (num 7) (num 1))))))
```

`N` はASTには出てこない。前処理の段階で `7` に置き換えられているためである。ASTから見ると、最初から `7` と書いてあったのと同じになる。

図1 `define_min.c` の AST

4 複数ファイルのコンパイル

第 14 回までは、プログラムは 1 つの `.c` ファイルに収まっていた。第 15 回では、複数の `.c` ファイルを読み込み、すべての AST を 1 つにまとめてアセンブリを出力する。

4.1 実際の C コンパイラの手順

実際の C コンパイラは、次の 4 段階で処理を行う。

段階	作業	出力
preprocess	<code>#include</code> / <code>#define</code> を展開	前処理済み <code>.i</code> ファイル
compile	前処理済みソースをアセンブリに変換	<code>.s</code> ファイル
assemble	アセンブリをオブジェクトファイルに変換	<code>.o</code> ファイル
link	複数の <code>.o</code> ファイルを結合	実行ファイル

各 `.c` ファイルは別々にコンパイルされ、最後にリンカがまとめる。この仕組みにより、一部のファイルだけ再コンパイルすることができる。

4.2 今回の簡易方式

今回のコンパイラでは、複数ファイルを扱うにあたり、次の簡易的な方式をとる。

1. 指定された全 `.c` ファイルを読み込む
2. 各ファイルの AST を 1 つのリストにまとめる
3. 全体のグローバル変数を集める
4. 全体の文字列リテラルを集める
5. `.data` / `.bss` / `.text` を出力する

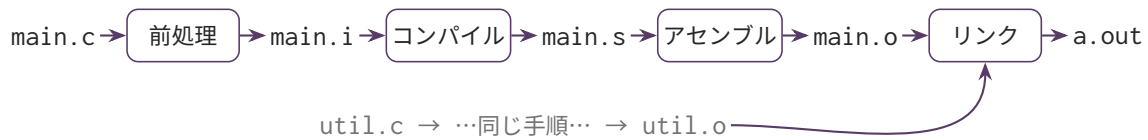
```
python3 mycc.py main.c math_util.c
```

この方式では、関数定義の重複チェックなどは行わない。また、実際の分割コンパイルのように、変更のあったファイルだけ再コンパイルする仕組みもない。しかし、複数ファイルに分割したプログラムを動かすことはできる。

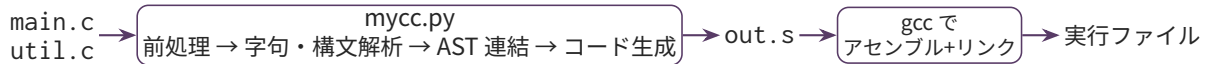
5 関数プロトタイプと定義

別ファイルにある関数を呼び出すには、呼び出し側のファイルで関数プロトタイプを宣言する必要がある。

実際の C コンパイラ（分割コンパイル）



今回の mycc.py（一括方式）



実際のコンパイラは .c ごとに .o を作り、変更したファイルだけ作り直せる。
mycc.py は毎回、全ファイルをまとめて 1 本のアセンブリとして出力する

図 2 実際の C コンパイラの分割コンパイルと、今回の一括方式

```

/* main.c */
int my_abs(int x);

int main() {
    return my_abs(-7);
}
  
```

関数プロトタイプ `int my_abs(int x);` は、AST 上は `'FuncProto'` ノードになる。これは「こういう関数がある」という宣言であり、コードは生成しない。実際のコードは `'FuncDef'` ノードだけが対象となる。

したがって、コンパイラのコード生成ループでは、`'FuncDef'` ノードだけを処理する。

```

for node in prog:
    if node.kind == 'FuncDef':
        self.gen_func(node)
  
```

6 実装手順

1. 第 14 回の実装を `sessions/15_preprocess_multifile/mycc.py` に反映する（スケルトンの `importlib` 継承により、前回の `Codegen14` クラスを継承する。新機能の handler だけを実装すればよい。）

2. `Codegen15.parse_file(filename)` クラスメソッドを導入し、読込・前処理・字句解析・構文解析を 1 つのクラスメソッドにまとめる
3. `main()` で、コマンドライン引数に指定された全 `.c` ファイルに対して `Codegen15.parse_file()` を呼び、AST リストを連結する
4. `self.collect_globals()` を全ファイルの AST に対して適用する
5. `self.collect_all_strings()` も全ファイルの AST に対して適用する
6. `.text` セクションでは `'FuncDef'` だけ进行处理する
7. `define_constants.c`、`multifile_math.c`、`multifile_global.c` を通す

7 編集するファイル

`sessions/15_preprocess_multifile/` 以下のスケルトンファイルを編集する。

ファイル	実装するハンドラ・メソッド
<code>mycc.py</code>	<code>Codegen15</code> クラス (<code>Codegen14</code> を継承)。 <code>parse_file()</code> クラスメソッド、 <code>collect_all_strings()</code> など
(AST パーサ)	変更不要 (<code>'FuncProto'</code> ノードは既に存在する)

8 テスト

```
python3 scaffold/test_runner.py sessions/15_preprocess_multifile
```

この回の主要テストは次の通り。

テスト	内容	期待値
<code>define_constants.c</code>	<code>#define</code> の定数置換	12
<code>multifile_math.c</code>	複数ファイル + <code>#include</code> + <code>#define</code>	64
<code>multifile_global.c</code>	複数ファイル + グローバル変数共有	63